【扔物线 Compose 课程讲义】3.7-derivedStateOf()——和 remember() 有什么区别?

derivedStateOf() 的作用

和带参数的 remember()的区别

小知识

扔物线学堂 - 秉心 (1437182)

关于讲义

讲义是对课程内容中**所有结论的集中汇总**。所以在看完每节课程之后，迅速来看一遍讲义，对课程内容做回顾，会有非常好的吸收效果。

面试前不要背讲义

在面试之前，通过快速浏览一遍讲义来确认所有知识都没有忘记，是个很好的备战方式。

但**不要用【背讲义上的知识点】的方式来备战**。最有效的浏览方式是：

浏览讲义中的所有内容，并：

- 看到「懂，并且记得背后的原因（或技术细节）」的，直接跳过；
- 看到「懂，但细节已经忘了」的，快速回看当节课程，把知识捡起来；
- 看到「已经不太懂 / 完全忘了」的，认真重看当节课程，把知识学回来。

疑问或意见

如果你发现讲义或课程有错误，或有别的什么意见，可以向任何一个你认识的助教反馈。

如果认可我们的课程，还请给个好评！ 评价链接：rengwuxian.com/cp

`derivedStateOf()` 的作用

当状态依赖别的一个或多个状态的时候，用 `derivedStateOf()` 来创建这个状态，可以在依赖的状态发生改变的时候去自动做重新计算。

和带参数的 `remember()` 的区别

- 带参数的 `remember()` 是在 Recompose 发生的时候去判断参数对象是否发生了「重新赋值」，而判断不出对象的内部状态的改变。因此对于「内部状态的监听」（例如 `mutableStateListOf()` ），只能用 `derivedStateOf()` 而不能用带参数的 `remember()` 。
- 函数参数的形式会阻断 `by mutableStateOf()` 所代理的对象的监听链条，所以在参数对象改变时， `derivedStateOf()` 无法自动监听到改变后的新的对象。因此对于「参数的对象替换」的监听（实际上是 Recompose 过程的判断而不是监听），需要用带参数的 `remember()` 。

因此， `derivedStateOf()` 的适用场景是「状态对象」，而带参数的 `remember()` 适合的场景是「函数参数」。二者其实并不是冲突互斥的关系。

- 另外，对于 `by mutableStateOf()` ，由于它的「状态改变」和「重新赋值」是一回事，所以用带参数的 `remember()` 和 `derivedStateOf()` 都行。
- 以及，如果状态对象成为了 Composable 函数的参数，就两个都用。

小知识

1. `derivedStateOf()` 和 `mutableStateOf()` 一样，并不是一个 Composable 函数，所以可以用在任何地方；
2. 由于 `derivedStateOf()` 创造的是一个状态对象，所以它在依赖的状态改变而导致重新计算并得出新值的时候，触发的是这个 `derivedStateOf()` 被使用处的 Recompose，而不是声明处的 Recompose。